Université de Rouen Normandie

Master 1 – Génie Informatique et Logiciel

Projet Gestion de Flotte

Rapport de Semaine 5

Service Événements & Alertes — Service Localisation

Étudiants :
Ahcene Amouchas
Florian Pepin

Encadrants :
Lydia & Luc

Année universitaire :
2025 – 2026

Semaine :
5 / 7

9 avril 2026

Table des matières

1 Introduction **2**
1.1 Répartition du travail 2

2 Service Événements & Alertes **3**
2.1 Modèle d'alerte 3
2.2 Consommation Kafka 3
2.3 API REST 3
2.4 Intégration GraphQL 4
2.5 Tests 4
2.6 Déploiement 4

3 Service Localisation **5**
3.1 Choix technologiques 5
3.2 Contrat gRPC 5
3.3 Détection de géofencing 5
3.4 Simulateur GPS 5
3.5 Déploiement 6

4 Tests Bruno **7**

5 Bilan et prochaines étapes **8**
5.1 Récapitulatif de la semaine 8
5.2 Semaine 6 8

1. Introduction

La semaine 5 ajoute deux nouveaux microservices au système : le **service événements**, qui centralise toutes les alertes de la flotte, et le **service localisation**, qui gère les positions GPS des véhicules en temps réel.

1.1. Répartition du travail

— **Florian Pepin** — Service événements & alertes : consommation Kafka, API REST, intégration GraphQL, tests, déploiement.

— **Ahcene Amouchas** — Service localisation : ingestion GPS via gRPC, stockage séries temporelles, géofencing, simulation.

Les deux services ont été développés en parallèle avec un contrat d'interface Kafka prédéfini, permettant à chacun d'avancer indépendamment.

2. Service Événements & Alertes

Rôle : Agrégateur central passif. Le service écoute les topics Kafka des autres microservices, transforme chaque événement pertinent en alerte persistante, et expose une API pour que les opérateurs puissent les consulter et les gérer.

2.1. Modèle d'alerte

Chaque alerte stocke un **type**, une **sévérité** et un **statut**, ainsi que des références logiques (UUID) vers le véhicule et le conducteur concernés — sans clé étrangère inter-services.

Type d'alerte	Source	Sévérité
`MAINTENANCE_OVERDUE`	Service Maintenance (Kafka)	WARNING / INFO
`LICENSE_EXPIRING`	Service Conducteurs (Kafka)	WARNING
`LICENSE_EXPIRED`	Service Conducteurs (Kafka)	CRITICAL
`GEOFENCING_BREACH`	Service Localisation (Kafka)	HIGH
`SPEED_EXCEEDED`	Service Localisation (Kafka)	WARNING à CRITICAL
`VEHICLE_IMMOBILIZED`	Service Localisation (Kafka)	WARNING

Table 1 – Types d'alertes supportés et leur origine

Le cycle de vie d'une alerte suit trois états : **ACTIVE** → **ACKNOWLEDGED** → **RESOLVED**. Une alerte peut aussi être résolue directement sans passer par l'état acquitté.

2.2. Consommation Kafka

Le service s'abonne à trois topics :

— **`flotte.alertes.events`** : messages publiés par le service maintenance lors d'un retard de maintenance. Chaque message produit une alerte `MAINTENANCE_OVERDUE`.

— **`flotte.conducteurs.events`** : messages publiés par le service conducteurs. Seuls les événements `LICENSE_EXPIRING` et `LICENSE_EXPIRED` génèrent une alerte ; les autres (`DRIVER_CREATED`, etc.) sont ignorés silencieusement.

— **`flotte.location.events`** : consumer déjà déployé et prêt à recevoir les événements du service localisation (`GEOFENCING_BREACH`, `SPEED_EXCEEDED`...). Il attend silencieusement les premiers messages sans erreur.

Un choix technique notable : tous les messages sont désérialisés manuellement (déserialiseur String brut + Jackson), ce qui rend le service compatible avec n'importe quel producteur quel que soit son langage (Java, Node.js, etc.) sans dépendance aux en-têtes propriétaires de Spring Kafka.

2.3. API REST

L'ensemble des routes est exposé sous le préfixe `/api/alerts` :

Méthode	Endpoint	Description
GET	`/api/alerts`	Liste avec filtres (statut, sévérité, type, vehicleId...)
GET	`/api/alerts/{id}`	Détail d'une alerte
GET	`/api/alerts/stats`	Nombre d'alertes actives
PATCH	`/api/alerts/{id}/acknowledge`	Acquitter une alerte (admin, manager)
PATCH	`/api/alerts/{id}/resolve`	Résoudre une alerte (admin, manager, technicien)

Table 2 – Endpoints REST du service événements

L'identifiant de l'opérateur qui acquitte une alerte est extrait directement du jeton JWT Keycloak (claim `sub`), sans le passer dans le corps de la requête. Cela évite tout risque d'usurpation d'identité.

2.4. Intégration GraphQL

Les opérations `alerts`, `alert`, `acknowledgeAlert` et `resolveAlert`, jusqu'ici en état `NOT_IMPLEMENTED` dans la gateway, sont désormais branchées sur ce service.

Un point notable : lorsqu'une alerte est récupérée via GraphQL, les champs imbriqués `vehicle` et `driver` sont résolus automatiquement par la gateway en interrogeant les services correspondants en parallèle. Si l'un des services est indisponible, le champ renvoie `null` sans faire échouer la requête.

Opération GraphQL	Statut
`alerts(...)`	**OK**
`alert(id)`	**OK**
`acknowledgeAlert(id)`	**OK**
`resolveAlert(id)`	**OK**
`Alert.vehicle` (résolution croisée)	**OK**
`Alert.driver` (résolution croisée)	**OK**
Subscriptions WebSocket	**Semaine 7**

Table 3 – Opérations GraphQL du domaine alertes

2.5. Tests

- **9 tests unitaires** (JUnit 5 + Mockito) couvrant la logique métier de l'`AlertService` : création, acquittement, résolution, idempotence, cas d'erreur.
- **0 échec**, couverture > 80% sur la couche service.
- **Collection Bruno** : 9 requêtes REST et 6 requêtes GraphQL, toutes valides en environnement Minikube.

2.6. Déploiement

Le service est disponible dans les deux environnements cibles :

- **Docker Compose** : conteneur `flotte-events` sur le port 8083, dépendant de PostgreSQL (healthcheck), Kafka et Keycloak.
- **Kubernetes / Helm** : ajout dans la map `services` du chart `fleet-app` (Deployment, Service, probes de santé), création de `events_db` dans le script `initdb` du chart `fleet-infra`, route `/api/alerts` ajoutée à l'Ingress.

3. Service Localisation

> **Rôle :** Recevoir et stocker les positions GPS des véhicules en continu, détecter les franchissements de zones (géofencing), et publier les alertes correspondantes sur Kafka.

3.1. Choix technologiques

Contrairement aux autres microservices (Java / Spring Boot), le service localisation est développé en **Node.js avec NestJS (TypeScript)**. Ce choix se justifie par la nature du problème : l'ingestion GPS implique un très grand nombre de connexions simultanées à faible charge unitaire, ce que le modèle événementiel non-bloquant de Node.js gère bien mieux qu'un modèle thread-par-requête.

La communication entre les boitiers embarqués et le service utilise **gRPC** (Protocol Buffers) plutôt que HTTP/REST. gRPC permet d'établir un tunnel persistant entre le véhicule et le serveur, évitant de rouvrir une connexion TCP à chaque point GPS envoyé — ce qui est coûteux à haute fréquence.

Pour la persistance, **PostgreSQL** est utilisé avec l'extension **TimescaleDB**, qui convertit la table de positions en *hypertable* (partitionnée automatiquement par plage temporelle). Cela garantit des performances stables même avec des millions de points stockés.

3.2. Contrat gRPC

Le fichier `location.proto` définit deux méthodes principales :

- **`StreamPositions`** : streaming bidirectionnel. Le véhicule pousse ses coordonnées GPS (latitude, longitude, vitesse, cap) en continu dans un tunnel persistant ; le serveur acquitte la réception de chaque trame. C'est la méthode principale d'ingestion.
- **`WatchVehicle`** : streaming serveur (server-side streaming). Permet à un tableau de bord de s'abonner en temps réel aux mouvements d'un véhicule spécifique, via les Observables RxJS de NestJS.

3.3. Détection de géofencing

Des zones géographiques (*geofences*) sont définissables via l'API REST du service (`/api/geofences`). À chaque nouvelle position reçue, le service calcule si le véhicule se trouve en dehors d'une zone configurée. En cas de franchissement, un événement `GEOFENCING_BREACH` est publié sur le topic Kafka `flotte.location.events`, qui est immédiatement traité par le service événements pour créer une alerte.

3.4. Simulateur GPS

Afin de valider l'architecture sans véhicule physique, un **simulateur** a été développé en parallèle du service. Il s'agit d'un script Node.js autonome qui joue le rôle d'un client gRPC externe : il génère un itinéraire factice (départ de Rouen) et envoie des coordonnées GPS réalistes toutes les 3 secondes via `StreamPositions`.

Cette séparation physique (le simulateur est un processus distinct, pas un module interne) garantit que la validation se fait par le réseau, comme en production. Le simulateur est intégré à la stack Docker Compose et peut également servir de test de charge sur la base de données temporelle.

3.5. Déploiement

— **Docker Compose** : deux conteneurs ajoutés — `location` (NestJS, port 3000 HTTP + 50051 gRPC) et `location-simulator` (client gRPC autonome).

— **Kubernetes** : Deployment et Service appliqués manuellement (manifestes dédiés, hors chart `fleet-app` qui est conçu pour les services Java). La base `location_db` est créée dans le script `initdb` du chart `fleet-infra`. Les routes `/api/geofences` et `/api/locations` sont exposées via l'Ingress Nginx.

4. Tests Bruno

Contexte : La collection Bruno couvre l'intégralité des services déployés. Cette semaine, 18 nouvelles requêtes ont été ajoutées pour les deux nouveaux services : 9 REST et 6 GraphQL pour le service événements, 3 REST pour le service localisation.

Service	Requêtes REST	Requêtes GraphQL
service-vehicule	5	6
service-driver	8	4
service-maintenance	5	6
service-events	9 *(nouveau)*	6 *(nouveau)*
service-location	3 *(nouveau)*	—
Total	**30**	**22**

Table 4 – Répartition des 52 requêtes de la collection Bruno

L'exécution complète sur l'environnement Minikube donne les résultats suivants :

Métrique	Résultat
Statut global	**PASS**
Requêtes	53 / 53
Tests	81 / 81

Table 5 – Résultat de `bru run --env-file environments/Minikube.json -r`

5. Bilan et prochaines étapes

5.1. Récapitulatif de la semaine

Fonctionnalité	Service	Statut
Consommation Kafka (maintenance + conducteurs)	Événements	**OK**
Consumer Kafka prêt pour localisation	Événements	**OK**
API REST gestion des alertes	Événements	**OK**
Intégration GraphQL (queries + mutations)	Événements	**OK**
Tests unitaires (couverture > 80%)	Événements	**OK**
Ingestion GPS via gRPC	Localisation	**OK**
Stockage TimescaleDB (séries temporelles)	Localisation	**OK**
Détection géofencing + publication Kafka	Localisation	**OK**
API REST géofences	Localisation	**OK**
Simulateur GPS (Docker Compose)	Localisation	**OK**
Déploiement Minikube (deux services)	Infrastructure	**OK**
Tests Bruno : 53 requêtes, 81/81 tests	Infrastructure	**OK**
Subscriptions WebSocket (alertes temps réel)	Événements	**S7**

Table 6 – Bilan de la semaine 5

5.2. Semaine 6

La semaine 6 sera consacrée au développement du **front-end** : mise en place de l'interface utilisateur permettant aux opérateurs de visualiser et gérer la flotte (véhicules, conducteurs, alertes, carte de localisation).